

Модуль 1: Основы языка C# и объектно-ориентированное программирование

1. Система типов данных в Csharp. Значимые (value) и ссылочные (reference) типы.
2. Процессы упаковки (boxing) и распаковки (unboxing). Их влияние на производительность.
3. Жизненный цикл объекта и механизм сборки мусора (Garbage Collection) в .NET.
4. Поколения в сборщике мусора и метод Finalize.
5. Интерфейс IDisposable и конструкция using. Управление неуправляемыми ресурсами.
6. Делегаты: понятие, объявление, использование. Типы Action, Func, Predicate.
7. События (events) и паттерн «Издатель-Подписчик».
8. Анонимные методы и лямбда-выражения. Замыкания (closures).
9. Обобщения (generics): принципы работы, ограничения (constraints), ковариантность и контравариантность.
10. Принципы объектно-ориентированного программирования: инкапсуляция, наследование, полиморфизм.
11. Абстрактные классы и интерфейсы. Сравнение и сценарии использования.
12. Модификаторы доступа в C# (public, private, protected, internal, protected internal, private protected).
13. Статические классы, методы и поля. Особенности их использования.
14. Конструкторы: статические, экземпляра, цепочки вызовов (this, base).
15. Принципы SOLID в контексте языка C#.
16. Обработка исключений: блоки try, catch, finally, throw. Создание пользовательских исключений.
17. Рефлексия (Reflection): получение метаданных, динамическое создание экземпляров.
18. LINQ (Language Integrated Query): синтаксис запросов и синтаксис методов. Отложенное выполнение (deferred execution).
19. Сопоставление с образцом (Pattern Matching): выражения is, switch.
20. Операторы ?. , ??, ??= для безопасной работы с null-ссылками.
21. Структуры (struct): отличия от классов, особенности размещения в памяти.
22. Перечисления (enum) и работа с флагами ([Flags]).
23. Неявно типизированные локальные переменные (var).
24. Пространства имен (namespaces) и директивы using. Глобальные пространства имен.

Модуль 2: Структуры данных и алгоритмы на C#

1. Асимптотическая сложность алгоритмов. Нотация Big-O, Omega и Theta.
2. Массивы: особенности реализации в памяти, преимущества и недостатки.
3. Класс `List<T>`: внутренняя реализация, динамическое изменение размера, сложность операций.
4. Связные списки (`LinkedList<T>`): односвязные и двусвязные списки, сложность вставки и удаления.
5. Стек (`Stack<T>`): принцип LIFO, основные операции и сценарии применения.
6. Очередь (`Queue<T>`): принцип FIFO, основные операции и сценарии применения.
7. Хеш-таблицы: принцип работы, хеш-функции, разрешение коллизий (цепочки, открытая адресация).
8. Класс `Dictionary<TKey, TValue>`: внутреннее устройство, сложность операций поиска, вставки и удаления.
9. Класс `HashSet<T>`: особенности реализации и отличия от `Dictionary`.
10. Деревья: основные понятия (корень, узел, лист, высота, глубина).
11. Графы: способы представления.
12. Алгоритмы обхода графа: поиск в ширину (BFS) и поиск в глубину (DFS).
13. Алгоритм сортировки слиянием (Merge Sort): принцип работы и оценка сложности.
14. Алгоритм быстрой сортировки (Quick Sort): выбор опорного элемента, оценка сложности.
15. Бинарный поиск: принцип работы, требования к данным, оценка сложности.
16. Рекурсия: понятие, базовый случай, рекурсивный случай. Переполнение стека.
17. Жадные алгоритмы: принцип работы, примеры задач.
18. Алгоритм Дейкстры для поиска кратчайшего пути во взвешенном графе.

Модуль 3: Разработка десктопных приложений с использованием WPF

1. Синтаксис XAML: пространства имен, свойства, вложенные элементы.
2. Паттерн проектирования MVVM (Model-View-ViewModel): назначение и разделение ответственности.
3. Интерфейс `INotifyPropertyChanged`: реализация и роль в обновлении UI.
4. Интерфейс `ICommand` и реализация команд.
5. Привязка данных (Data Binding): режимы (OneWay, TwoWay, OneTime, OneWayToSource).
6. Контекст данных (`DataContext`): наследование и установка.

7. Ресурсы в WPF: `StaticResource` и `DynamicResource`. Области видимости ресурсов.
8. Стили (Style): создание, применение, триггеры в стилях, наследование стилей (BasedOn).
9. Шаблоны элементов управления (ControlTemplate): перерисовка визуальной структуры контрола.
10. Шаблоны данных (DataTemplate): отображение объектов данных в интерфейсе.
11. Панели компоновки: `Grid` (определение строк и столбцов, *, Auto).
12. Панели компоновки: `StackPanel`, `WrapPanel`. Особенности рендеринга.
13. Элементы управления для работы с коллекциями: `ListBox`, `ComboBox`, `ListView`.
14. Создание пользовательских элементов управления.
15. Файл `App.xaml`: ресурсы уровня приложения, события `Startup` и `Exit`.
16. Навигация в WPF: `NavigationWindow`, `Frame`, `Page`.

Модуль 4: Формальные языки и трансляторы

1. Иерархия Хомского: типы 0, 1, 2, 3 и соответствующие им автоматы.
2. Регулярные языки и регулярные выражения.
3. Детерминированные конечные автоматы (DFA): определение, функция переходов, диаграмма состояний.
4. Недетерминированные конечные автоматы (NFA) и NFA с ϵ -переходами.
5. Алгоритм построения DFA из NFA.
6. Контекстно-свободные грамматики (КС-грамматики): терминалы, нетерминалы, правила вывода.
7. Нормальная форма Бэкуса-Наура (БНФ).
8. Автоматы с магазинной памятью (Pushdown Automata): устройство и принцип работы.
9. Машина Тьюринга: формальное определение, универсальная машина Тьюринга.
10. Этапы работы компилятора.
11. Лексический анализ.
12. Синтаксический анализ (парсинг): задачи и классификация методов (нисходящие и восходящие).
13. Метод рекурсивного спуска: принцип реализации и обработка левой рекурсии.
14. Таблица имён.
15. Генерация объектного (машинного) кода.
16. Оптимизация кода.
17. Сравнение компилятора и интерпретатора.

18. Однопроходные и многопроходные компиляторы: преимущества и недостатки.

Модуль 5: Клиент-серверная архитектура и сетевые протоколы

1. Протоколы транспортного уровня: сравнение TCP и UDP.
2. Протокол HTTP: структура запроса и ответа, заголовки.
3. Протокол HTTPS: принцип работы, рукопожатие TLS/SSL, сертификаты.
4. Архитектурный стиль REST: основные принципы.
5. Идемпотентность HTTP-методов: GET, POST, PUT, PATCH, DELETE.
6. Коды состояния HTTP: классы 1xx, 2xx, 3xx, 4xx, 5xx и примеры.
7. Форматы обмена данными.
8. Протокол WebSocket: механизм установления соединения (handshake)
9. Понятие stateful и stateless серверов.
10. Аутентификация и авторизация: Basic Auth, Session-based, Token-based (JWT).
11. Механизм CORS (Cross-Origin Resource Sharing): preflight-запросы, заголовки.
12. Паттерн API Gateway: назначение, маршрутизация, агрегация запросов.
13. Монолитная архитектура vs Микросервисная архитектура: плюсы, минусы, критерии выбора.
14. Масштабируемость: горизонтальное (scale-out) и вертикальное (scale-up) масштабирование.
15. Пагинация, фильтрация и сортировка данных в REST API (query parameters).

Модуль 6: Контейнеризация: Docker и docker-compose

1. Понятие контейнеризации. Отличия контейнеров от виртуальных машин.
2. Архитектура Docker.
3. Понятие образа (Image): слои, неизменяемость, идентификаторы.
4. Понятие контейнера (Container).
5. Инструкция FROM в Dockerfile.
6. Инструкции RUN, CMD и ENTRYPOINT: различия и сценарии использования.
7. Инструкции COPY и ADD: функциональные различия и рекомендации по применению.
8. Директива WORKDIR: назначение и влияние на последующие инструкции.
9. Работа с переменными окружения: инструкции ENV и ARG, их отличия.
10. Механизм кэширования слоев при сборке образа. Правила инвалидации кэша.
11. Многоэтапная сборка (Multi-stage builds): оптимизация размера финального образа.

12. Команды управления образами.
13. Команды управления контейнерами.
14. Просмотр логов контейнера.
15. Выполнение команд внутри работающего контейнера.
16. Типы хранения данных: анонимные тома, именованные тома (Named Volumes).
17. Структура файла `docker-compose.yml`: версии, сервисы, сети, тома.
18. Понятие сервиса в Docker Compose. Масштабирование сервисов (`--scale`).
19. Директива `depends_on`.
20. Использование файлов переменных окружения (`.env`) в `docker-compose`.
21. Переопределение конфигурации с помощью файла `docker-compose.override.yml`.

Модуль 7: Проектирование программных систем: UML и BPMN

1. Назначение и классификация UML-диаграмм: структурные и поведенческие.
2. Диаграмма прецедентов (Use Case Diagram): актеры, прецеденты, система.
3. Диаграмма классов (Class Diagram): классы, атрибуты, методы, видимость.
4. Отношения в диаграмме классов: ассоциация, агрегация, композиция, наследование (генерализация), зависимость.
5. Диаграмма компонентов (Component Diagram): компоненты, интерфейсы, зависимости.
6. Диаграмма развертывания (Deployment Diagram): узлы (nodes), артефакты, каналы связи.
7. Диаграмма последовательности (Sequence Diagram): объекты, линии жизни, сообщения (синхронные, асинхронные).
8. Диаграмма состояний (State Machine Diagram): состояния, переходы, события, действия.
9. Назначение и область применения нотации BPMN 2.0.
10. Основные элементы BPMN: События (Events) – начальные, промежуточные, конечные.
11. Основные элементы BPMN: Действия (Activities) – задачи (Tasks) и подпроцессы (Sub-Processes).
12. Основные элементы BPMN: Шлюзы (Gateways) – эксклюзивный (XOR), параллельный (AND), включающий (OR).
13. Потoki управления (Sequence Flow) и потоки сообщений (Message Flow) в BPMN.
14. Пулы (Pools) и дорожки (Lanes) в BPMN: моделирование взаимодействия между организациями и ролями.

15. Артефакты в BPMN: объекты данных (Data Object), хранилища данных, аннотации.
16. Принципы проектирования ПО: DRY (Don't Repeat Yourself), KISS (Keep It Simple, Stupid), YAGNI (You Aren't Gonna Need It).
17. Принцип единственной ответственности (SRP) на уровне модулей и классов.
18. Основы проектирования баз данных: ER-диаграммы (сущности, атрибуты, связи).
19. Нормализация реляционных баз данных: первая (1НФ), вторая (2НФ) и третья (3НФ) нормальные формы.
20. Документирование API с использованием спецификации OpenAPI (Swagger).